

Timing Test: Direct vs Cascaded ST Pipelines

This notebook measures per-clip latency for the direct and cascaded speech-translation pipelines on the same 35 pilot clips. The purpose is feasibility testing for the eventual 4,200-sample run.

1. Purpose

This timing notebook is intentionally focused on computational feasibility, not translation quality. It measures per-clip runtime, records failures, and estimates the wall-clock cost of the full experiment.

```
In [1]: from __future__ import annotations

import json
import platform
import sys
import time
from pathlib import Path

%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torchaudio
from tqdm.auto import tqdm
from transformers import AutoModelForSeq2SeqLM, AutoProcessor, AutoTokenizer, SeamlessM4Tv2ForSpeechToText, pipeline

# PROJECT_ROOT is one directory above the notebook directory.
PROJECT_ROOT = Path.cwd().resolve()
print(f'Project root resolved to: {PROJECT_ROOT}')

CSV_PATH = PROJECT_ROOT / 'data' / 'timing_test_audio' / 'timing_test_35_clips.csv'
CLIP_DIR = PROJECT_ROOT / 'data' / 'timing_test_audio' / 'clips'
OUTPUTS_DIR = PROJECT_ROOT / 'outputs' / 'timing'
FIGURES_DIR = PROJECT_ROOT / 'results' / 'figures'
OUTPUTS_DIR.mkdir(parents=True, exist_ok=True)
FIGURES_DIR.mkdir(parents=True, exist_ok=True)

if not CSV_PATH.exists():
    raise FileNotFoundError(
        'Could not find timing_test_35_clips.csv at project_root/data/timing_test_audio/. '
        f'Checked: {CSV_PATH}. '
        'Ensure this notebook is run from the notebooks directory.'
    )
if not CLIP_DIR.exists():
    raise FileNotFoundError(
        'Could not find clips directory at project_root/data/timing_test_audio/clips. '
        f'Checked: {CLIP_DIR}. '
        'Ensure this notebook is run from the notebooks directory.'
    )

# Fixed target language (no config lookup).
TARGET_LANGUAGE = 'zh'
MODEL_DIRECT = 'facebook/seamless-m4t-v2-large'
MODEL_ASR = 'openai/whisper-large-v2'
MODEL_MT = 'facebook/nllb-200-distilled-600M'

print(f'Project root resolved to: {PROJECT_ROOT}')
print(f'CSV path: {CSV_PATH}')
print(f'Clip directory: {CLIP_DIR}')
print(f'Target language: {TARGET_LANGUAGE}')
print(f'Direct model: {MODEL_DIRECT}')
print(f'ASR model: {MODEL_ASR}')
print(f'NLLB model: {MODEL_MT}')
```

```
Project root resolved to: /content
Project root resolved to: /content
CSV path: /content/data/timing_test_audio/timing_test_35_clips.csv
Clip directory: /content/data/timing_test_audio/clips
Target language: zh
Direct model: facebook/seamless-m4t-v2-large
ASR model: openai/whisper-large-v2
NLLB model: facebook/nllb-200-distilled-600M
```

2. Imports and Configuration

This section sets project paths, resolves the target language if a config exists, and prepares the libraries required for runtime logging and plotting.

```
In [2]: def get_gpu_summary():
    if not torch.cuda.is_available():
        return {'name': None, 'total_memory_gb': None}
    try:
        props = torch.cuda.get_device_properties(torch.cuda.current_device())
        return {'name': props.name, 'total_memory_gb': round(float(props.total_memory / (1024 ** 3)), 2)}
    except Exception:
```

```
return {'name': None, 'total_memory_gb': None}
```

```
gpu = get_gpu_summary()
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
environment = {
    'timestamp_utc': time.strftime('%Y-%m-%dT%H:%M:%SZ', time.gmtime()),
    'python_version': sys.version.replace('\n', ' '),
    'torch_version': torch.__version__,
    'transformers_version': __import__('transformers').__version__,
    'cuda_available': torch.cuda.is_available(),
    'cuda_version': torch.version.cuda if torch.cuda.is_available() else None,
    'gpu_name': gpu['name'],
    'gpu_memory_gb': gpu['total_memory_gb'],
    'device': str(device),
    'model_direct': MODEL_DIRECT,
    'model_asr': MODEL_ASR,
    'model_mt': MODEL_MT,
    'target_language': TARGET_LANGUAGE,
    'platform_system': platform.system(),
    'platform_release': platform.release(),
    'platform_version': platform.version(),
    'platform_machine': platform.machine(),
    'notebook_timing_methodology': 'time.perf_counter() with torch.cuda.synchronize() before and after each GPU inference call; n
}
for key, value in environment.items():
    print(f'{key}: {value}')

with (OUTPUTS_DIR / 'timing_test_environment.json').open('w', encoding='utf-8') as f:
    json.dump(environment, f, indent=2, sort_keys=True)
```

```
timestamp_utc: 2026-08-22T03:07:21Z
python_version: 3.13.15 (main, Aug 6 2026, 11:06:23) [GCC 11.4.0]
torch_version: 2.11.0+cu128
transformers_version: 5.15.0
cuda_available: True
cuda_version: 12.8
gpu_name: Tesla T4
gpu_memory_gb: 14.56
device: cuda
model_direct: facebook/seamless-m4t-v2-large
model_asr: openai/whisper-large-v2
model_mt: facebook/nllb-200-distilled-600M
target_language: zh
platform_system: Linux
platform_release: 6.6.122+
platform_version: #1 SMP Thu Apr 30 18:17:14 UTC 2026
platform_machine: x86_64
notebook_timing_methodology: time.perf_counter() with torch.cuda.synchronize() before and after each GPU inference call; model lo
ading time is excluded from each clip measurement.
```

3. Hardware and Environment Information

Because runtime is hardware-dependent, the notebook records detailed environment metadata before the timing loop begins.

```
In [3]: required_columns = [
    'client_id', 'path', 'sentence_id', 'sentence', 'sentence_domain', 'up_votes', 'down_votes',
    'age', 'gender', 'accents', 'variant', 'locale', 'segment', 'primary_accent',
    'sentence_norm', 'clip', 'duration[ms]', 'sentence_len_words', 'max_plausible_words',
    'sentence_len_chars'
]

df = pd.read_csv(CSV_PATH)
missing = [c for c in required_columns if c not in df.columns]
assert len(df) == 35, f'Expected exactly 35 rows, found {len(df)}'
assert not missing, f'Missing required columns: {missing}'

df['sample_id'] = df.apply(lambda r: f"{r['sentence_id']}_{r['client_id']}", axis=1)
seen = {}
for idx, row in df.iterrows():
    key = row['sample_id']
    if key in seen:
        seen[key] += 1
        df.at[idx, 'sample_id'] = f"{key}_{seen[key]}"
    else:
        seen[key] = 0

df['audio_path'] = df['clip'].map(lambda x: str(CLIP_DIR / str(x)))
audio_valid = df['audio_path'].map(lambda p: Path(p).exists())
missing_audio = df.loc[~audio_valid, ['sample_id', 'clip', 'audio_path']]
assert audio_valid.all(), f'Missing audio files: {missing_audio.to_dict('records')}'
assert df['sample_id'].is_unique, 'sample_id must be unique'
assert pd.to_numeric(df['duration[ms]'], errors='coerce').notna().all(), 'duration[ms] contains invalid values'
assert df['primary_accent'].notna().all(), 'primary_accent is missing in one or more rows'

print(f'Total rows: {len(df)}')
print(f'Valid audio files: {int(audio_valid.sum())}')
print(f'Missing audio files: {int((~audio_valid).sum())}')
print('Accent counts:')
print(df['primary_accent'].value_counts().to_string())
print(f'Mean audio duration (ms): {df['duration[ms]'].mean():.1f}')
print(f'Min audio duration (ms): {df['duration[ms]'].min():.1f}')
print(f'Max audio duration (ms): {df['duration[ms]'].max():.1f}')
print(f'Mean sentence length in words: {df['sentence_len_words'].mean():.2f}')
```

```
df['audio_duration_sec'] = df['duration[ms]'] / 1000.0
display(df[['sample_id', 'client_id', 'sentence_id', 'primary_accent', 'clip', 'audio_duration_sec', 'sentence_len_words']].head(
```

```
Total rows: 35
Valid audio files: 35
Missing audio files: 0
Accent counts:
primary_accent
England English          5
Filipino                 5
Hong Kong English       5
India and South Asia (India, Pakistan, Sri Lanka) 5
Malaysian English       5
Southern African (South Africa, Zimbabwe, Namibia) 5
United States English   5
Mean audio duration (ms): 4651.2
Min audio duration (ms): 2232.0
Max audio duration (ms): 8424.0
Mean sentence length in words: 9.14
```

	sample_id	client_id
0	1f4e17468b0ae46e0bc22be117c073f7d85e54f15a3a0b...	d22f218cb2c44e73fdb321fef4dc52bc8995d28048330b... 1f4e17468b0ae46e0bc22be117c073
1	2c54888ad9e686725a29e6b7888e6b0cf27198d9a46c23...	48f48da505573df2ae0b591a7d6b7ace44cc94ac5838f6... 2c54888ad9e686725a29e6b7888e6b0
2	0bd200f6b026ead6064433d4ea9a0cbbedd5d1a351f205...	46bb908b70bc8a4e4512ff78304e1f53aff78f86e34e2b... 0bd200f6b026ead6064433d4ea9a0c
3	e4a0940769b7aa2773bd4ea62064ec73acb1e43a6afa1a...	fe4168562cc5f0320bc378cb946838be67dbe6808773ea... e4a0940769b7aa2773bd4ea62064ec
4	01ac98c06562fbb5deb692ca12f66769dc6f799b4bbc47...	16ca33c77a4ff48f931fe63a9986938db81dc24e4be8eb... 01ac98c06562fbb5deb692ca12f6676

4. Load and Validate 35-Clip Timing Dataset

The CSV is validated before timing begins, including row count, required columns, valid audio paths, unique sample IDs, valid durations, and accent information.

5. Dataset Duration and Accent Summary

```
In [4]: print('Mean audio duration (sec):', round(float(df['audio_duration_sec'].mean()), 3))
print('Median audio duration (sec):', round(float(df['audio_duration_sec'].median()), 3))
print('Total audio duration (sec):', round(float(df['audio_duration_sec'].sum()), 3))
print('Mean sentence length in words:', round(float(df['sentence_len_words'].mean()), 2))
```

```
Mean audio duration (sec): 4.651
Median audio duration (sec): 4.296
Total audio duration (sec): 162.792
Mean sentence length in words: 9.14
```

6. Timing Methodology

Each clip is measured individually with `time.perf_counter()`. If CUDA is in use, we call `torch.cuda.synchronize()` immediately before and after the inference call to account for asynchronous GPU execution.

```
In [5]: def sync_if_cuda():
        if torch.cuda.is_available():
            torch.cuda.synchronize()

def timed_inference(fn, *args, **kwargs):
    sync_if_cuda()
    t0 = time.perf_counter()
    with torch.inference_mode():
        result = fn(*args, **kwargs)
    sync_if_cuda()
    elapsed = time.perf_counter() - t0
    return result, elapsed
```

7. Load Models

```
In [6]: try:
        direct_processor = AutoProcessor.from_pretrained(MODEL_DIRECT)
        direct_model = SeamlessM4Tv2ForSpeechToText.from_pretrained(MODEL_DIRECT)
        direct_model.to(device)
        direct_model.eval()

        asr_pipe = pipeline(
            'automatic-speech-recognition',
            model=MODEL_ASR,
            device=0 if torch.cuda.is_available() else -1,
            model_kwargs={'torch_dtype': torch.float16 if torch.cuda.is_available() else torch.float32},
        )

        nllb_tokenizer = AutoTokenizer.from_pretrained(MODEL_MT, src_lang='eng_Latn')
        nllb_model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_MT)
        nllb_model.to(device)
        nllb_model.eval()
        print('All required models loaded.')
    except Exception as exc:
```

```
print(f'Model loading failed: {exc}')
direct_processor = None
direct_model = None
asr_pipe = None
nllb_tokenizer = None
nllb_model = None
```

/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_auth.py:138: UserWarning:

Error while fetching `HF\_TOKEN` secret value from your vault: 'Requesting secret HF_TOKEN timed out. Secrets can only be fetched when running from the Colab UI.'

warnings.warn(f"\nError while fetching `HF\_TOKEN` secret value from your vault: '{str(e)}'."

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
preprocessor_config.json: 0%|          | 0.00/1.78k [00:00<?, ?B/s]
config.json: 0%|          | 0.00/2.72k [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/19.7k [00:00<?, ?B/s]
tokenizer.model: reconstructing file: 0%|          | 0.00B / 5.17MB
tokenizer.model: downloading bytes:          | 0.00B
added_tokens.json: 0%|          | 0.00/2.07k [00:00<?, ?B/s]
special_tokens_map.json: 0%|          | 0.00/2.34k [00:00<?, ?B/s]
model.safetensors.index.json: 0%|          | 0.00/211k [00:00<?, ?B/s]
Downloading bytes:          | 0.00B
Reconstructing (incomplete total...): |          | 0.00B / 0.00B
Fetching 2 files: 0%|          | 0/2 [00:00<?, ?it/s]
Loading weights: 0%|          | 0/1429 [00:00<?, ?it/s]
```

Key	Status	
text_encoder.layers.{0...23}.self_attn.k_proj.bias	UNEXPECTED	
vocoder.hifi_gan.resblocks.{0...14}.convs2.{0, 1, 2}.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.ffn.fc1.weight	UNEXPECTED	
vocoder.hifi_gan.resblocks.{0...14}.convs1.{0, 1, 2}.bias	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.out_proj.bias	UNEXPECTED	
text_encoder.layers.{0...23}.self_attn_layer_norm.bias	UNEXPECTED	
text_encoder.layers.{0...23}.self_attn.out_proj.bias	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.out_proj.bias	UNEXPECTED	
vocoder.hifi_gan.resblocks.{0...14}.convs2.{0, 1, 2}.bias	UNEXPECTED	
text_encoder.layers.{0...23}.ffn_layer_norm.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn_layer_norm.weight	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.conv1.bias	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.q_proj.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.ffn_layer_norm.weight	UNEXPECTED	
text_encoder.layers.{0...23}.self_attn.v_proj.bias	UNEXPECTED	
vocoder.dur_predictor.conv1.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.v_proj.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn_layer_norm.bias	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.k_proj.bias	UNEXPECTED	
text_encoder.layers.{0...23}.ffn.fc2.bias	UNEXPECTED	
vocoder.hifi_gan.resblocks.{0...14}.convs1.{0, 1, 2}.weight	UNEXPECTED	
text_encoder.layers.{0...23}.self_attn.q_proj.bias	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.conv_layer_norm.bias	UNEXPECTED	
text_encoder.layers.{0...23}.self_attn.q_proj.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.ffn_layer_norm.bias	UNEXPECTED	
text_encoder.layers.{0...23}.self_attn.v_proj.weight	UNEXPECTED	
text_encoder.layers.{0...23}.ffn.fc1.weight	UNEXPECTED	
text_encoder.layers.{0...23}.ffn_layer_norm.bias	UNEXPECTED	
text_encoder.layers.{0...23}.ffn.fc1.bias	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.q_proj.bias	UNEXPECTED	
text_encoder.layers.{0...23}.self_attn_layer_norm.weight	UNEXPECTED	
vocoder.hifi_gan.upsampler.{0, 1, 2, 3, 4}.bias	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.v_proj.weight	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.q_proj.bias	UNEXPECTED	
text_encoder.layers.{0...23}.self_attn.k_proj.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.out_proj.weight	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.conv2.bias	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.conv2.weight	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.conv_layer_norm.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.ffn.fc1.bias	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.out_proj.weight	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.conv1.weight	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.v_proj.bias	UNEXPECTED	
text_encoder.layers.{0...23}.self_attn.out_proj.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.ffn.fc2.bias	UNEXPECTED	
vocoder.hifi_gan.upsampler.{0, 1, 2, 3, 4}.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.q_proj.weight	UNEXPECTED	
vocoder.dur_predictor.proj.weight	UNEXPECTED	
t2u_model.model.encoder.layer_norm.bias	UNEXPECTED	
vocoder.hifi_gan.conv_pre.bias	UNEXPECTED	
vocoder.dur_predictor.conv1.bias	UNEXPECTED	
text_encoder.layers.{0...23}.ffn.fc2.weight	UNEXPECTED	
vocoder.speaker_embedding.weight	UNEXPECTED	
t2u_model.model.decoder.embed_tokens.weight	UNEXPECTED	
vocoder.dur_predictor.conv2.bias	UNEXPECTED	
t2u_model.model.decoder.pos_emb_alpha	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.k_proj.bias	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.v_proj.bias	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn_layer_norm.bias	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.k_proj.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.ffn.fc2.weight	UNEXPECTED	
vocoder.dur_predictor.ln1.weight	UNEXPECTED	
text_encoder.layer_norm.weight	UNEXPECTED	
t2u_model.model.encoder.layers.{0, 1, 2, 3, 4, 5}.self_attn.k_proj.weight	UNEXPECTED	
t2u_model.model.encoder.layer_norm.weight	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.ln1.bias	UNEXPECTED	
t2u_model.model.decoder.layers.{0, 1, 2, 3, 4, 5}.self_attn_layer_norm.weight	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.ln2.weight	UNEXPECTED	
vocoder.dur_predictor.ln1.bias	UNEXPECTED	
t2u_model.model.decoder.embed_char.weight	UNEXPECTED	
vocoder.unit_embedding.weight	UNEXPECTED	
vocoder.dur_predictor.proj.bias	UNEXPECTED	
vocoder.hifi_gan.conv_post.bias	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.ln1.weight	UNEXPECTED	
vocoder.dur_predictor.conv2.weight	UNEXPECTED	
vocoder.hifi_gan.conv_pre.weight	UNEXPECTED	
vocoder.dur_predictor.ln2.weight	UNEXPECTED	
t2u_model.model.decoder.layer_norm.bias	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.conv2.bias	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.conv1.weight	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.ln2.bias	UNEXPECTED	
vocoder.hifi_gan.conv_post.weight	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.conv2.weight	UNEXPECTED	
text_encoder.layer_norm.bias	UNEXPECTED	
vocoder.language_embedding.weight	UNEXPECTED	
t2u_model.model.decoder.layer_norm.weight	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.proj.weight	UNEXPECTED	
vocoder.dur_predictor.ln2.bias	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.conv1.bias	UNEXPECTED	
t2u_model.model.decoder.duration_predictor.proj.bias	UNEXPECTED	
t2u_model.model.decoder.pos_emb_alpha_char	UNEXPECTED	

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

generation_config.json: 0%| | 0.00/9.91M [00:00<?, ?B/s]

config.json: 0%| | 0.00/1.99k [00:00<?, ?B/s]

[transformers] `torch\_dtype` is deprecated! Use `dtype` instead!

[transformers] `torch\_dtype` is deprecated! Use `dtype` instead!

model.safetensors: reconstructing file: 0%| | 0.00B / 6.17GB

model.safetensors: downloading bytes: | 0.00B

Loading weights: 0%| | 0/1259 [00:00<?, ?it/s]

generation_config.json: 0%| | 0.00/4.29k [00:00<?, ?B/s]

tokenizer_config.json: 0%| | 0.00/283k [00:00<?, ?B/s]

vocab.json: 0%| | 0.00/836k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/2.48M [00:00<?, ?B/s]

merges.txt: 0%| | 0.00/494k [00:00<?, ?B/s]

normalizer.json: 0%| | 0.00/52.7k [00:00<?, ?B/s]

added_tokens.json: 0%| | 0.00/34.6k [00:00<?, ?B/s]

special_tokens_map.json: 0%| | 0.00/2.19k [00:00<?, ?B/s]

preprocessor_config.json: 0%| | 0.00/185k [00:00<?, ?B/s]

config.json: 0%| | 0.00/846 [00:00<?, ?B/s]

tokenizer_config.json: 0%| | 0.00/564 [00:00<?, ?B/s]

sentencepiece.bpe.model: reconstructing file: 0%| | 0.00B / 4.85MB

sentencepiece.bpe.model: downloading bytes: | 0.00B

tokenizer.json: reconstructing file: 0%| | 0.00B / 17.3MB

tokenizer.json: downloading bytes: | 0.00B

special_tokens_map.json: 0%| | 0.00/3.55k [00:00<?, ?B/s]

pytorch_model.bin: reconstructing file: 0%| | 0.00B / 2.46GB

pytorch_model.bin: downloading bytes: | 0.00B

Loading weights: 0%| | 0/512 [00:00<?, ?it/s]

model.safetensors: reconstructing file: 0%| | 0.00B / 2.46GB

model.safetensors: downloading bytes: | 0.00B

generation_config.json: 0%| | 0.00/189 [00:00<?, ?B/s]

All required models loaded.

8. Warm-Up

In [7]: SEAMLESS_SAMPLE_RATE = 16000

```
def load_waveform(audio_path, target_sr: int = SEAMLESS_SAMPLE_RATE):
    waveform, sr = torchaudio.load(audio_path)
    # Convert multi-channel audio to mono to keep processor input consistent.
    if waveform.ndim == 2 and waveform.size(0) > 1:
        waveform = waveform.mean(dim=0, keepdim=True)
    # Resample to the rate required by SeamlessM4T feature extractor.
    if sr != target_sr:
        waveform = torchaudio.functional.resample(waveform, sr, target_sr)
        sr = target_sr
    waveform = waveform.squeeze(0).contiguous()
    return waveform, sr

def direct_translate_single(audio_path: str, processor, model, target_lang: str):
    waveform, sr = load_waveform(audio_path)
    inputs = processor(audio=waveform.numpy(), sampling_rate=sr, return_tensors='pt')
    input_features = inputs['input_features'].to(device)
    attention_mask = inputs.get('attention_mask')
    if attention_mask is not None:
        attention_mask = attention_mask.to(device)
    seamless_target_lang = 'cmn' if target_lang == 'zh' else target_lang
    generated = model.generate(
        input_features,
        attention_mask=attention_mask,
        tgt_lang=seamless_target_lang,
    )
    return processor.batch_decode(generated, skip_special_tokens=True)[0]

def asr_single(audio_path: str):
    if asr_pipe is None:
        raise RuntimeError('ASR pipeline not loaded')
    result = asr_pipe(audio_path, return_timestamps=False, max_new_tokens=128)
    if isinstance(result, dict):
        return result.get('text', '')
    if isinstance(result, list):
        return result[0].get('text', '') if result else ''
    return str(result)

def nllb_translate_single(text: str, tokenizer, model, target_lang_code: str):
    if tokenizer is None or model is None:
        raise RuntimeError('NLLB model or tokenizer not available')
    inputs = tokenizer(text, return_tensors='pt', truncation=True)
    inputs = {k: v.to(device) for k, v in inputs.items()}
    nllb_target_lang = {'zh': 'zho_Hans'}.get(target_lang_code, target_lang_code)
    target_token = tokenizer.convert_tokens_to_ids(nllb_target_lang)
    if target_token is None or target_token == tokenizer.unk_token_id:
        raise ValueError(f'Unsupported NLLB target language code: {nllb_target_lang!r}')
    generated = model.generate(**inputs, forced_bos_token_id=target_token)
    return tokenizer.decode(generated[0], skip_special_tokens=True)

warmup_path = df.iloc[0]['audio_path']
if direct_model is not None and direct_processor is not None:
    _, _ = timed_inference(direct_translate_single, warmup_path, direct_processor, direct_model, TARGET_LANGUAGE)
    print('Direct warm-up complete')
if asr_pipe is not None and nllb_model is not None and nllb_tokenizer is not None:
```



```

asr_text, _ = timed_inference(asr_single, warmup_path)
_, _ = timed_inference(nllb_translate_single, asr_text, nllb_tokenizer, nllb_model, TARGET_LANGUAGE)
print('Cascade warm-up complete')

```

[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer SeamlessM4TTokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True to force cleanup anyway.

Direct warm-up complete

[transformers] Passing `generation\_config` together with generation-related arguments=({'max_new_tokens'}) is deprecated and will be removed in future versions. Please pass either a `generation\_config` object OR all generation parameters explicitly, but not both.

[transformers] Using custom `forced\_decoder\_ids` from the (generation) config. This is deprecated in favor of the `task` and `language` flags/config options.

[transformers] Transcription using a multilingual Whisper will default to language detection followed by transcription instead of translation to English. This might be a breaking change for your use case. If you want to instead always translate your audio to English, make sure to pass `language='en'`. See <https://github.com/huggingface/transformers/pull/28687> for more details.

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[transformers] A custom logits processor of type <class 'transformers.generation.logits_process.SuppressTokensLogitsProcessor'> has been passed to `.generate()`, but it was also created in `.generate()`, given its parameterization. The custom <class 'transformers.generation.logits_process.SuppressTokensLogitsProcessor'> will take precedence. Please check the docstring of <class 'transformers.generation.logits_process.SuppressTokensLogitsProcessor'> to see related `.generate()` flags.

[transformers] A custom logits processor of type <class 'transformers.generation.logits_process.SuppressTokensAtBeginLogitsProcessor'> has been passed to `.generate()`, but it was also created in `.generate()`, given its parameterization. The custom <class 'transformers.generation.logits_process.SuppressTokensAtBeginLogitsProcessor'> will take precedence. Please check the docstring of <class 'transformers.generation.logits_process.SuppressTokensAtBeginLogitsProcessor'> to see related `.generate()` flags.

[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer WhisperTokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True to force cleanup anyway.

Cascade warm-up complete

9. Direct Pipeline Timing

```

In [8]: direct_rows = []
for idx, row in tqdm(df.iterrows(), total=len(df), desc='Direct pipeline'):
    rec = {
        'sample_id': row['sample_id'],
        'client_id': row['client_id'],
        'sentence_id': row['sentence_id'],
        'primary_accent': row['primary_accent'],
        'clip': row['clip'],
        'audio_path': row['audio_path'],
        'duration_ms': row['duration[ms]'],
        'audio_duration_sec': row['audio_duration_sec'],
        'sentence_len_words': row['sentence_len_words'],
        'sentence_len_chars': row['sentence_len_chars'],
        'direct_translation': None,
        'direct_runtime_sec': None,
        'direct_realtime_factor': None,
        'direct_status': 'failed',
        'direct_error': None,
    }
    try:
        if direct_model is None or direct_processor is None:
            raise RuntimeError('Direct model not loaded')
        translation, runtime = timed_inference(direct_translate_single, row['audio_path'], direct_processor, direct_model, TARGET_LANGUAGE)
        rec['direct_translation'] = translation
        rec['direct_runtime_sec'] = float(runtime)
        rec['direct_realtime_factor'] = float(runtime) / float(row['audio_duration_sec']) if row['audio_duration_sec'] > 0 else None
        rec['direct_status'] = 'success'
        print(f"[Direct] {idx + 1:02d}/{len(df)} | accent={row['primary_accent']} | audio={row['audio_duration_sec']:.2f}s | runtime={rec['direct_runtime_sec']:.2f}s")
    except Exception as exc:
        rec['direct_error'] = repr(exc)
        print(f"[Direct] {idx + 1:02d}/{len(df)} FAILED | accent={row['primary_accent']} | error={exc}")
    direct_rows.append(rec)

direct_df = pd.DataFrame(direct_rows)
display(direct_df.head())

```

Direct pipeline: 0%| | 0/35 [00:00<?, ?it/s]

[Direct]	01/35		accent=England	English		audio=5.21s		runtime=1.0917s		RTF=0.21
[Direct]	02/35		accent=England	English		audio=3.79s		runtime=3.3033s		RTF=0.87
[Direct]	03/35		accent=England	English		audio=4.90s		runtime=5.7536s		RTF=1.18
[Direct]	04/35		accent=England	English		audio=8.42s		runtime=6.1054s		RTF=0.72
[Direct]	05/35		accent=England	English		audio=4.66s		runtime=1.9539s		RTF=0.42
[Direct]	06/35		accent=Filipino			audio=5.52s		runtime=3.6835s		RTF=0.67
[Direct]	07/35		accent=Filipino			audio=7.46s		runtime=1.1398s		RTF=0.15
[Direct]	08/35		accent=Filipino			audio=6.79s		runtime=4.6429s		RTF=0.68
[Direct]	09/35		accent=Filipino			audio=6.12s		runtime=0.9494s		RTF=0.16
[Direct]	10/35		accent=Filipino			audio=3.12s		runtime=5.7475s		RTF=1.84
[Direct]	11/35		accent=Hong Kong	English		audio=3.34s		runtime=1.0474s		RTF=0.31
[Direct]	12/35		accent=Hong Kong	English		audio=4.30s		runtime=0.9042s		RTF=0.21
[Direct]	13/35		accent=Hong Kong	English		audio=2.26s		runtime=0.6481s		RTF=0.29
[Direct]	14/35		accent=Hong Kong	English		audio=3.34s		runtime=0.8372s		RTF=0.25
[Direct]	15/35		accent=Hong Kong	English		audio=6.79s		runtime=1.1591s		RTF=0.17
[Direct]	16/35		accent=India and South Asia	(India, Pakistan, Sri Lanka)		audio=2.23s		runtime=0.6866s		RTF=0.31
[Direct]	17/35		accent=India and South Asia	(India, Pakistan, Sri Lanka)		audio=3.34s		runtime=0.9038s		RTF=0.27
[Direct]	18/35		accent=India and South Asia	(India, Pakistan, Sri Lanka)		audio=7.34s		runtime=1.3643s		RTF=0.19
[Direct]	19/35		accent=India and South Asia	(India, Pakistan, Sri Lanka)		audio=6.58s		runtime=1.4775s		RTF=0.22
[Direct]	20/35		accent=India and South Asia	(India, Pakistan, Sri Lanka)		audio=3.98s		runtime=0.8519s		RTF=0.21
[Direct]	21/35		accent=Malaysian	English		audio=2.78s		runtime=0.7840s		RTF=0.28
[Direct]	22/35		accent=Malaysian	English		audio=2.98s		runtime=0.7620s		RTF=0.26
[Direct]	23/35		accent=Malaysian	English		audio=3.29s		runtime=0.8542s		RTF=0.26
[Direct]	24/35		accent=Malaysian	English		audio=3.10s		runtime=0.8039s		RTF=0.26
[Direct]	25/35		accent=Malaysian	English		audio=5.50s		runtime=0.9285s		RTF=0.17
[Direct]	26/35		accent=Southern African	(South Africa, Zimbabwe, Namibia)		audio=2.54s		runtime=0.6774s		RTF=0.27
[Direct]	27/35		accent=Southern African	(South Africa, Zimbabwe, Namibia)		audio=2.64s		runtime=0.7187s		RTF=0.27
[Direct]	28/35		accent=Southern African	(South Africa, Zimbabwe, Namibia)		audio=4.87s		runtime=0.8476s		RTF=0.17
[Direct]	29/35		accent=Southern African	(South Africa, Zimbabwe, Namibia)		audio=7.90s		runtime=1.3064s		RTF=0.17
[Direct]	30/35		accent=Southern African	(South Africa, Zimbabwe, Namibia)		audio=4.39s		runtime=0.7693s		RTF=0.18
[Direct]	31/35		accent=United States	English		audio=3.55s		runtime=0.8398s		RTF=0.24
[Direct]	32/35		accent=United States	English		audio=4.46s		runtime=1.2089s		RTF=0.27
[Direct]	33/35		accent=United States	English		audio=7.75s		runtime=1.8923s		RTF=0.24
[Direct]	34/35		accent=United States	English		audio=3.82s		runtime=0.8805s		RTF=0.23
[Direct]	35/35		accent=United States	English		audio=3.74s		runtime=0.7654s		RTF=0.20

	sample_id	client_id
0	1f4e17468b0ae46e0bc22be117c073f7d85e54f15a3a0b...	d22f218cb2c44e73fdb321fef4dc52bc8995d28048330b... 1f4e17468b0ae46e0bc22be117c073
1	2c54888ad9e686725a29e6b7888e6b0cf27198d9a46c23...	48f48da505573df2ae0b591a7d6b7ace44cc94ac5838f6... 2c54888ad9e686725a29e6b7888e6b0
2	0bd200f6b026ead6064433d4ea9a0cbbbedd5d1a351f205...	46bb908b70bc8a4e4512ff78304e1f53aff78f86e34e2b... 0bd200f6b026ead6064433d4ea9a0c
3	e4a0940769b7aa2773bd4ea62064ec73acb1e43a6afa1a...	fe4168562cc5f0320bc378cb946838be67dbe6808773ea... e4a0940769b7aa2773bd4ea62064ec
4	01ac98c06562fbb5deb692ca12f66769dc6f799b4bbc47...	16ca33c77a4ff48f931fe63a9986938db81dc24e4be8eb... 01ac98c06562fbb5deb692ca12f6676

10. Cascaded Pipeline Timing

```
In [9]: cascade_rows = []
for idx, row in tqdm(df.iterrows(), total=len(df), desc='Cascade pipeline'):
    rec = {
        'sample_id': row['sample_id'],
        'client_id': row['client_id'],
        'sentence_id': row['sentence_id'],
        'primary_accent': row['primary_accent'],
        'clip': row['clip'],
        'audio_path': row['audio_path'],
        'duration_ms': row['duration[ms]'],
        'audio_duration_sec': row['audio_duration_sec'],
        'sentence_len_words': row['sentence_len_words'],
        'sentence_len_chars': row['sentence_len_chars'],
        'asr_transcript': None,
        'whisper_runtime_sec': None,
        'cascaded_translation': None,
        'nllb_runtime_sec': None,
        'cascade_runtime_sec': None,
        'cascade_end_to_end_runtime_sec': None,
        'cascade_realtime_factor': None,
        'cascade_status': 'failed',
        'cascade_error': None,
    }
    try:
        if asr_pipe is None or nllb_model is None or nllb_tokenizer is None:
            raise RuntimeError('Cascade models not loaded')
        asr_text, whisper_runtime = timed_inference(asr_single, row['audio_path'])
        rec['asr_transcript'] = asr_text
        rec['whisper_runtime_sec'] = float(whisper_runtime)
        nllb_text, nllb_runtime = timed_inference(nllb_translate_single, asr_text, nllb_tokenizer, nllb_model, TARGET_LANGUAGE)
        rec['cascaded_translation'] = nllb_text
        rec['nllb_runtime_sec'] = float(nllb_runtime)
        rec['cascade_runtime_sec'] = float(rec['whisper_runtime_sec']) + float(rec['nllb_runtime_sec'])
        rec['cascade_end_to_end_runtime_sec'] = rec['cascade_runtime_sec']
```



```
rec['cascade_reftime_factor'] = float(rec['cascade_runtime_sec']) / float(row['audio_duration_sec']) if row['audio_duration_sec'] > 0:
rec['cascade_status'] = 'success'
print(f"[Cascade] {idx + 1:02d}/{len(df)} | accent={row['primary_accent']} | audio={row['audio_duration_sec']:.2f}s | Whisper={row['whisper_runtime_sec']:.2f}s | NLLB={row['nllb_runtime_sec']:.2f}s | total={row['total_runtime_sec']:.2f}s | RTF={row['rtf']}")
except Exception as exc:
rec['cascade_error'] = repr(exc)
print(f"[Cascade] {idx + 1:02d}/{len(df)} FAILED | accent={row['primary_accent']} | error={exc}")
cascade_rows.append(rec)
```

```
cascade_df = pd.DataFrame(cascade_rows)
display(cascade_df.head())
```

Cascade pipeline: 0% | 0/35 [00:00<?, ?it/s]

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 01/35 | accent=England English | audio=5.21s | Whisper=1.7817s | NLLB=0.2549s | total=2.0366s | RTF=0.39

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 02/35 | accent=England English | audio=3.79s | Whisper=1.0639s | NLLB=0.2037s | total=1.2676s | RTF=0.33

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 03/35 | accent=England English | audio=4.90s | Whisper=1.3300s | NLLB=0.2687s | total=1.5987s | RTF=0.33

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 04/35 | accent=England English | audio=8.42s | Whisper=1.1522s | NLLB=0.1757s | total=1.3279s | RTF=0.16

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 05/35 | accent=England English | audio=4.66s | Whisper=1.3385s | NLLB=0.4433s | total=1.7818s | RTF=0.38

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 06/35 | accent=Filipino | audio=5.52s | Whisper=1.5666s | NLLB=0.3255s | total=1.8922s | RTF=0.34

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 07/35 | accent=Filipino | audio=7.46s | Whisper=1.3985s | NLLB=0.2183s | total=1.6169s | RTF=0.22

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 08/35 | accent=Filipino | audio=6.79s | Whisper=1.4855s | NLLB=0.2546s | total=1.7401s | RTF=0.26

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[transformers] You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a dataset

[Cascade] 09/35 | accent=Filipino | audio=6.12s | Whisper=1.4347s | NLLB=0.2323s | total=1.6670s | RTF=0.27

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 10/35 | accent=Filipino | audio=3.12s | Whisper=0.9570s | NLLB=0.1319s | total=1.0889s | RTF=0.35

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 11/35 | accent=Hong Kong English | audio=3.34s | Whisper=1.0873s | NLLB=0.1987s | total=1.2860s | RTF=0.39

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 12/35 | accent=Hong Kong English | audio=4.30s | Whisper=1.0964s | NLLB=0.1854s | total=1.2818s | RTF=0.30

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 13/35 | accent=Hong Kong English | audio=2.26s | Whisper=0.8623s | NLLB=0.1065s | total=0.9688s | RTF=0.43

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 14/35 | accent=Hong Kong English | audio=3.34s | Whisper=1.3057s | NLLB=0.2562s | total=1.5619s | RTF=0.47

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 15/35 | accent=Hong Kong English | audio=6.79s | Whisper=1.7659s | NLLB=0.3310s | total=2.0970s | RTF=0.31

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 16/35 | accent=India and South Asia (India, Pakistan, Sri Lanka) | audio=2.23s | Whisper=0.9033s | NLLB=0.1209s | total=1.0242s | RTF=0.46

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 17/35 | accent=India and South Asia (India, Pakistan, Sri Lanka) | audio=3.34s | Whisper=1.1423s | NLLB=0.2162s | total=1.3585s | RTF=0.41

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 18/35 | accent=India and South Asia (India, Pakistan, Sri Lanka) | audio=7.34s | Whisper=1.2170s | NLLB=0.2791s | total=1.4961s | RTF=0.20

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 19/35 | accent=India and South Asia (India, Pakistan, Sri Lanka) | audio=6.58s | Whisper=1.2941s | NLLB=0.2358s | total=1.5300s | RTF=0.23

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 20/35 | accent=India and South Asia (India, Pakistan, Sri Lanka) | audio=3.98s | Whisper=1.2013s | NLLB=0.2131s | total=1.4144s | RTF=0.36

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 21/35 | accent=Malaysian English | audio=2.78s | Whisper=0.9618s | NLLB=0.1571s | total=1.1189s | RTF=0.40

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 22/35 | accent=Malaysian English | audio=2.98s | Whisper=1.1923s | NLLB=0.2143s | total=1.4067s | RTF=0.47

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 23/35 | accent=Malaysian English | audio=3.29s | Whisper=1.3265s | NLLB=0.2569s | total=1.5834s | RTF=0.48

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 24/35 | accent=Malaysian English | audio=3.10s | Whisper=1.2224s | NLLB=0.2143s | total=1.4366s | RTF=0.46

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 25/35 | accent=Malaysian English | audio=5.50s | Whisper=1.1083s | NLLB=0.1471s | total=1.2554s | RTF=0.23

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 26/35 | accent=Southern African (South Africa, Zimbabwe, Namibia) | audio=2.54s | Whisper=0.8582s | NLLB=0.1200s | total=0.9782s | RTF=0.38

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 27/35 | accent=Southern African (South Africa, Zimbabwe, Namibia) | audio=2.64s | Whisper=0.9105s | NLLB=0.1195s | total=1.0301s | RTF=0.39

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 28/35 | accent=Southern African (South Africa, Zimbabwe, Namibia) | audio=4.87s | Whisper=1.1523s | NLLB=0.1862s | total=1.3385s | RTF=0.27

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 29/35 | accent=Southern African (South Africa, Zimbabwe, Namibia) | audio=7.90s | Whisper=1.5686s | NLLB=0.3479s | total=1.9165s | RTF=0.24

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 30/35 | accent=Southern African (South Africa, Zimbabwe, Namibia) | audio=4.39s | Whisper=1.3169s | NLLB=0.1466s | total=1.4635s | RTF=0.33

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 31/35 | accent=United States English | audio=3.55s | Whisper=1.0161s | NLLB=0.1388s | total=1.1549s | RTF=0.33

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 32/35 | accent=United States English | audio=4.46s | Whisper=1.1108s | NLLB=0.2740s | total=1.3848s | RTF=0.31

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 33/35 | accent=United States English | audio=7.75s | Whisper=2.1031s | NLLB=0.3312s | total=2.4343s | RTF=0.31

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 34/35 | accent=United States English | audio=3.82s | Whisper=1.3130s | NLLB=0.2112s | total=1.5241s | RTF=0.40

[transformers] Both `max\_new\_tokens` (=128) and `max\_length` (=448) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Cascade] 35/35 | accent=United States English | audio=3.74s | Whisper=1.1692s | NLLB=0.1453s | total=1.3146s | RTF=0.35

	sample_id	client_id
0	1f4e17468b0ae46e0bc22be117c073f7d85e54f15a3a0b...	d22f218cb2c44e73fdb321fef4dc52bc8995d28048330b... 1f4e17468b0ae46e0bc22be117c073
1	2c54888ad9e686725a29e6b7888e6b0cf27198d9a46c23...	48f48da505573df2ae0b591a7d6b7ace44cc94ac5838f6... 2c54888ad9e686725a29e6b7888e6b0
2	0bd200f6b026ead6064433d4ea9a0cbbedd5d1a351f205...	46bb908b70bc8a4e4512ff78304e1f53aff78f86e34e2b... 0bd200f6b026ead6064433d4ea9a0c
3	e4a0940769b7aa2773bd4ea62064ec73acb1e43a6afa1a...	fe4168562cc5f0320bc378cb946838be67dbe6808773ea... e4a0940769b7aa2773bd4ea62064ec
4	01ac98c06562fbb5deb692ca12f66769dc6f799b4bbc47...	16ca33c77a4ff48f931fe63a9986938db81dc24e4be8eb... 01ac98c06562fbb5deb692ca12f6676

11. Combine and Save Timing Results

```
In [10]: df = df.merge(direct_df[['sample_id', 'direct_translation', 'direct_runtime_sec', 'direct_realtime_factor', 'direct_status', 'direct_sentence_id', 'direct_path'], on='sample_id')
df = df.merge(cascade_df[['sample_id', 'asr_transcript', 'whisper_runtime_sec', 'cascaded_translation', 'nllb_runtime_sec', 'cascade_runtime_sec'], on='sample_id')
df['preferred_cascade_runtime_sec'] = df['cascade_end_to_end_runtime_sec'].fillna(df['cascade_runtime_sec'])
df['runtime_difference_sec'] = df['preferred_cascade_runtime_sec'] - df['direct_runtime_sec']

result_path = OUTPUTS_DIR / 'timing_test_35_clips_results.csv'
df.to_csv(result_path, index=False)
try:
    df.to_parquet(OUTPUTS_DIR / 'timing_test_35_clips_results.parquet', index=False)
except Exception as exc:
    print(f'Parquet export skipped: {exc}')
print(f'Results saved to {result_path}')
display(df.head())
```

Results saved to /content/outputs/timing/timing_test_35_clips_results.csv

	client_id	path	sentence_id	s
0	d22f218cb2c44e73fdb321fef4dc52bc8995d28048330b...	common_voice_en_19695528.mp3	1f4e17468b0ae46e0bc22be117c073f7d85e54f15a3a0b...	t nu s
1	48f48da505573df2ae0b591a7d6b7ace44cc94ac5838f6...	common_voice_en_20002436.mp3	2c54888ad9e686725a29e6b7888e6b0cf27198d9a46c23...	5
2	46bb908b70bc8a4e4512ff78304e1f53aff78f86e34e2b...	common_voice_en_685452.mp3	0bd200f6b026ead6064433d4ea9a0cbbedd5d1a351f205...	w
3	fe4168562cc5f0320bc378cb946838be67dbe6808773ea...	common_voice_en_536327.mp3	e4a0940769b7aa2773bd4ea62064ec73acb1e43a6afa1a...	se
4	16ca33c77a4ff48f931fe63a9986938db81dc24e4be8eb...	common_voice_en_18711143.mp3	01ac98c06562fbb5deb692ca12f66769dc6f799b4bbc47...	A 1

5 rows x 39 columns

```
In [11]: sample_results_table = df[['sentence', 'accents', 'direct_translation', 'cascaded_translation', 'preferred_cascade_runtime_sec', 'direct_runtime_sec']].copy()

sample_results_table[['preferred_cascade_runtime_sec', 'direct_runtime_sec']] = (
    sample_results_table[['preferred_cascade_runtime_sec', 'direct_runtime_sec']].round(4)
```

```
)
sample_results_table = sample_results_table.rename(columns={
    'cascaded_translation': 'cascade_translation',
    'preferred_cascade_runtime_sec': 'cascade_runtime',
    'direct_runtime_sec': 'direct_runtime',
})

with pd.option_context('display.max_rows', 35, 'display.max_colwidth', 200):
    display(sample_results_table)
```

	sentence	accents	direct_translation	cascade_translation	cascade_runtime	direct_runtime
0	In fact, the street numbering system is based on the Grid plan.	England English	事实上,街道编号系统是基于网格计划的.	实际上,街道编号系统是基于网格计划的.	2.0366	1.0917
1	The crew never speak his name again.	England English	船员们再也没有说他的名字了.	团队从来没有说过他的名字.	1.2676	3.3033
2	The men climbed the hill, and they were tired when they reached the top.	England English	这些人爬上了山,到达山顶时他们已经疲<unk>不堪.	那些人爬上山,到顶峰时,他们很累.	1.5987	5.7536
3	No one seemed to have thought about it that night.	England English	那天晚上似乎没有人想到这一点.	那天晚上似乎没有人想过.	1.3279	6.1054
4	A song by the same name by country music artist Chris LeDoux.	England English Northern England	这首歌由乡村音乐艺术家克里斯·莱杜克斯 (Chris LeDoux) 创作.	乡村音乐艺术家克里斯·莱多克斯的同名歌曲.	1.7818	1.9539
5	Sorry, I'll explain more when I actually understand what's going on.	Filipino	很抱歉,当我真正理解发生了什么时,我会解释更多.	我会解释更多,当我真正明白了什么.	1.8922	3.6835
6	The course of the battle is differently related in the contemporaneous historical records.	Filipino	战役的过程在当代历史记录中有所不同.	现代历史记录中,战斗的进程是不同的.	1.6169	1.1398
7	The seventeen-year-old girl gently pushed his hand away as he touched her belly.	Filipino	十七岁的女孩轻轻地推开他的手,因为他触摸她的肚子.	这位17岁的女孩轻轻地把他手推开,	1.7401	4.6429
8	Without knowing where they went, there's not much we can do but wait.	Filipino	没有知道他们去哪里,我们只能等待.	没有知道他们去哪里,我们只能等待.	1.6670	0.9494
9	YOU WANNA TAKE THIS OUTSIDE?	Filipino	你想把这个带出去吗?	你想出去看看吗?	1.0889	5.7475
10	The box was thrown beside the parked truck.	Hong Kong English	盒子被扔在停着的卡车旁边.	箱子被扔在车辆旁边.	1.2860	1.0474
11	Tea in thin china has a sweet taste.	Hong Kong English	薄<unk>的茶有甜的味道.	在薄薄的中国,茶有甜味.	1.2818	0.9042
12	She had no temperature.	Hong Kong English	她没有发烧	她没有发热.	0.9688	0.6481
13	He was ludicrously fast on a bike.	Hong Kong English	他骑着自行车的速度非常快.	他骑着自行车是荒谬的快速.	1.5619	0.8372
14	After the long night, I am happy to see that my phone still has a charge.	Hong Kong English	在漫长的夜晚之后,我很高兴看到我的手机还充电了.	经过漫长的夜晚,我很高兴看到我的手机还充电了.	2.0970	1.1591
15	You know very well why.	India and South Asia (India, Pakistan, Sri Lanka)	你非常清楚为什么.	你知道,为什么?	1.0242	0.6866
16	The model can be initialized with sparse coding.	India and South Asia (India, Pakistan, Sri Lanka)	该模型可以通过稀疏编码进行初始化.	模型可以以稀疏的编码启动.	1.3585	0.9038
17	He appeared six times each with John Wayne and Paul Newman.	India and South Asia (India, Pakistan, Sri Lanka)	他与约翰·韦恩和保罗·纽曼一起出现了六次.	他与约翰·韦恩和保罗·纽曼一起出现在六次.	1.4961	1.3643
18	Second, having more choices leads to an escalation of expectation.	India and South Asia (India, Pakistan, Sri Lanka)	其次,有更多的选择会导致预期的升级.	第二,有更多选择会导致人们的期望升级.	1.5300	1.4775
19	It is hard or even impossible to define the fitness expression.	India and South Asia (India, Pakistan, Sri Lanka)	很难甚至不可能定义适应度表达式	身体健康表达是很难甚至是不可能的.	1.4144	0.8519
20	Luke, I am your sister!	Malaysian English	卢克,我是你的妹妹!	卢克,我是你的妹妹.	1.1189	0.7840
21	Everyone seemed very excited.	Malaysian English	每个人似乎都非常兴奋.	所有的色都看起来很.	1.4067	0.7620
22	Yeah, I was in a wheel chair for months.	Malaysian English	是的,我坐轮椅了几个月	是的,我坐轮椅了几个月.	1.5834	0.8542
23	Fine, I agree this could be the case.	Malaysian English	好吧,我同意这可能是个情况.	很好,我同意这可能是这样的情况.	1.4366	0.8039
24	The Hindu Zunbils were also located here.	Malaysian English	印度教的Zunbils也位于这里.	印度尸也在此.	1.2554	0.9285
25	How did she look?	Southern African (South Africa, Zimbabwe, Namibia)	她是什么样子的?	她看起来怎么样?	0.9782	0.6774
26	You know what I want.	Southern African (South Africa, Zimbabwe, Namibia)	你知道我想要什么.	你知道我想要什么.	1.0301	0.7187
27	A force equal to that would move the earth.	Southern African (South Africa, Zimbabwe, Namibia)	一个相当于它的力量会使地球移动.	像这样的力量会动动地球.	1.3385	0.8476
28	The resulting settlement, called Bytown in his honour, was later renamed after the river.	Southern African (South Africa, Zimbabwe, Namibia)	结果的定居点,以他的名字命名为拜敦,后来以河的名字改名为拜敦.	后来的定居点,以他称之为拜敦,后来以河的名字改名.	1.9165	1.3064

	sentence	accents	direct_translation	cascade_translation	cascade_runtime	direct_runtime
29	'This is important, and you're going to have to do it.	Southern African (South Africa, Zimbabwe, Namibia)	这很重要,你必须这样做.	这很重要,你必须做.	1.4635	0.7693
30	The young boy dies from the impact.	United States English	这个小男孩死于撞击.	孩子死于撞击.	1.1549	0.8398
31	Three radio stations broadcast from Russell.	United States English	三个广播电台从拉塞尔播出.	美国三家广播公司从拉塞尔发射.	1.3848	1.2089
32	The New Latin term "corbicula" is a diminutive of "corbis", a basket or pannier.	United States English	新拉丁语"corbicula"一词是"corbis"的缩写,一个篮子或<unk>子.	拉丁语新词"corbicula"是"corbis"的缩写词,	2.4343	1.8923
33	He thought of all the married shepherds he had known.	United States English	他想起他认识的所有已婚牧羊人.	他想到了他认识的所有已婚牧民.	1.5241	0.8805
34	All you've got to do is talk to him.	United States English	你只需要和他谈谈.	你只需要跟他谈话.	1.3146	0.7654

12. Runtime Summary Statistics

In [12]:

```
def summarize(values):
    s = pd.Series(values).dropna()
    if s.empty:
        return {'successful': 0, 'failed': 0, 'mean': np.nan, 'median': np.nan, 'min': np.nan, 'max': np.nan, 'std': np.nan, 'total': 0}
    return {'successful': int(len(s)), 'failed': int(len(df) - len(s)), 'mean': float(s.mean()), 'median': float(s.median()), 'min': float(s.min()), 'max': float(s.max()), 'std': float(s.std()), 'total': int(len(df))}

direct_runtime_values = df['direct_runtime_sec'].dropna()
cascade_runtime_values = df['preferred_cascade_runtime_sec'].dropna()
direct_rtf_values = df['direct_realtime_factor'].dropna()
cascade_rtf_values = df['cascade_realtime_factor'].dropna()
whisper_runtime_values = df['whisper_runtime_sec'].dropna()
nllb_runtime_values = df['nllb_runtime_sec'].dropna()

print('Direct summary:', summarize(direct_runtime_values))
print('Cascade summary:', summarize(cascade_runtime_values))
print('Mean Whisper runtime:', whisper_runtime_values.mean())
print('Median Whisper runtime:', whisper_runtime_values.median())
print('Mean NLLB runtime:', nllb_runtime_values.mean())
print('Median NLLB runtime:', nllb_runtime_values.median())
print('Mean Direct RTF:', direct_rtf_values.mean())
print('Median Direct RTF:', direct_rtf_values.median())
print('Mean Cascade RTF:', cascade_rtf_values.mean())
print('Median Cascade RTF:', cascade_rtf_values.median())
print('Whisper percentage of Cascade runtime:', whisper_runtime_values.mean() / cascade_runtime_values.mean())
print('NLLB percentage of Cascade runtime:', nllb_runtime_values.mean() / cascade_runtime_values.mean())
print('Mean audio duration (sec):', df['audio_duration_sec'].mean())
print('Median audio duration (sec):', df['audio_duration_sec'].median())
print('Mean sentence length in words:', df['sentence_len_words'].mean())

Direct summary: {'successful': 35, 'failed': 0, 'mean': 1.665430926914275, 'median': 0.9284829869999385, 'min': 0.6480948119999539, 'max': 6.105438217000028, 'std': 1.5769888221916437, 'total': 58.29008244199963}
Cascade summary: {'successful': 35, 'failed': 0, 'mean': 1.4679059928571372, 'median': 1.4144314030000942, 'min': 0.9687648860000309, 'max': 2.434260543999926, 'std': 0.33413615492831655, 'total': 51.376709749999804}
Mean Whisper runtime: 1.2489782743999966
Median Whisper runtime: 1.2013197490000493
Mean NLLB runtime: 0.21892771845714054
Median NLLB runtime: 0.21425807900004656
Mean Direct RTF: 0.36580058977704466
Median Direct RTF: 0.2560335900537569
Mean Cascade RTF: 0.34145000404436426
Median Cascade RTF: 0.3427829480072398
Whisper percentage of Cascade runtime: 0.8508571260540881
NLLB percentage of Cascade runtime: 0.1491428739459118
Mean audio duration (sec): 4.651200000000001
Median audio duration (sec): 4.296
Mean sentence length in words: 9.142857142857142
```

13. Full 4,200-Sample Runtime Estimate

In []:

```
N_TOTAL = 4200
estimated_direct_mean = direct_runtime_values.mean() * N_TOTAL
estimated_direct_median = direct_runtime_values.median() * N_TOTAL
estimated_cascade_mean = cascade_runtime_values.mean() * N_TOTAL
estimated_cascade_median = cascade_runtime_values.median() * N_TOTAL
estimated_combined_mean = estimated_direct_mean + estimated_cascade_mean
estimated_combined_median = estimated_direct_median + estimated_cascade_median
print(f'Estimated Direct runtime for 4,200 clips (mean): {estimated_direct_mean / 3600:.2f} hours')
print(f'Estimated Direct runtime for 4,200 clips (median): {estimated_direct_median / 3600:.2f} hours')
print(f'Estimated Cascade runtime for 4,200 clips (mean): {estimated_cascade_mean / 3600:.2f} hours')
print(f'Estimated Cascade runtime for 4,200 clips (median): {estimated_cascade_median / 3600:.2f} hours')
print(f'Estimated combined runtime (mean): {estimated_combined_mean / 3600:.2f} hours')
print(f'Estimated combined runtime (median): {estimated_combined_median / 3600:.2f} hours')
print('This is a linear estimate and actual runtime can vary because of audio-duration distribution, GPU hardware, thermal throttling, etc')
```


Estimated Direct runtime for 4,200 clips (mean): 1.94 hours
 Estimated Cascade runtime for 4,200 clips (mean): 1.71 hours
 Estimated Cascade runtime for 4,200 clips (median): 1.65 hours
 Estimated combined runtime (mean): 3.66 hours
 Estimated combined runtime (median): 2.73 hours
 This is a linear estimate and actual runtime can vary because of audio-duration distribution, GPU hardware, thermal throttling, caching, model/device configuration, failed samples, batching, multiprocessing, or system load.

14. Runtime Visualisations

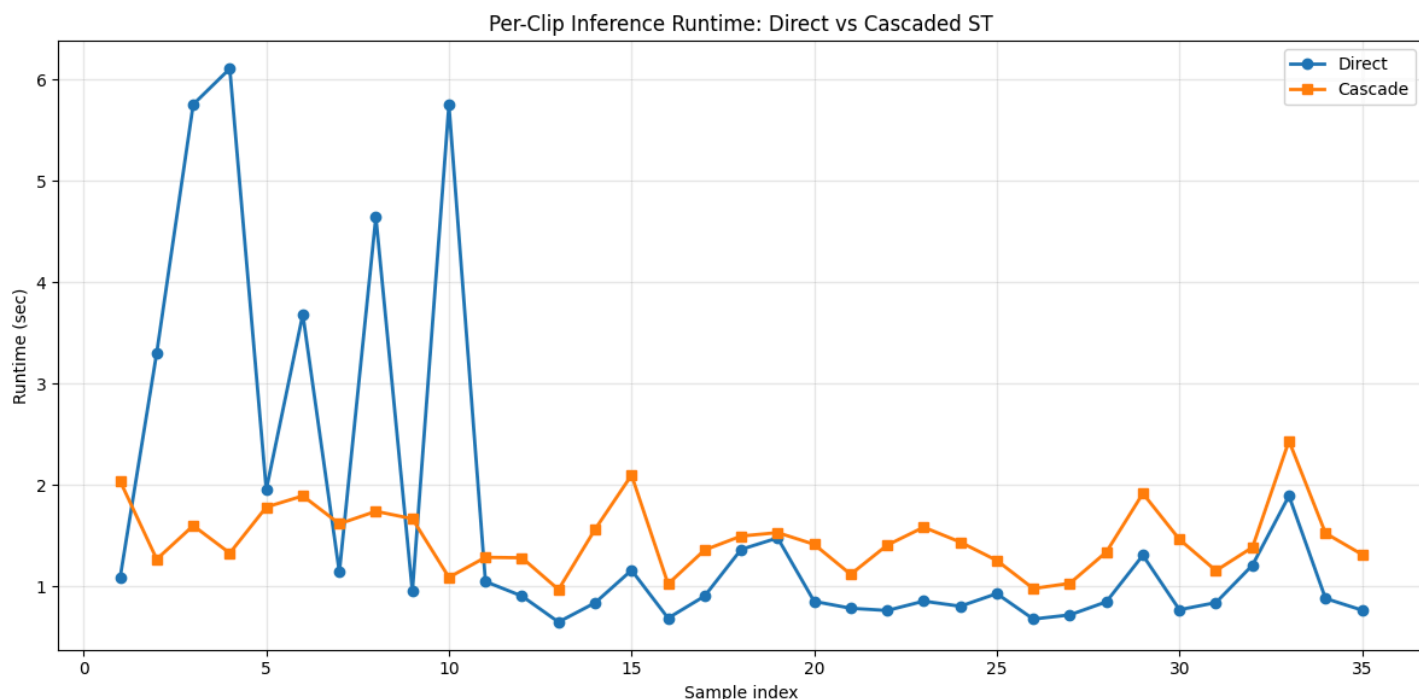
```
In [14]: sample_idx = np.arange(1, len(df) + 1)

plt.figure(figsize=(12, 6))
plt.plot(sample_idx, df['direct_runtime_sec'].fillna(0), marker='o', linewidth=2, label='Direct')
plt.plot(sample_idx, df['preferred_cascade_runtime_sec'].fillna(0), marker='s', linewidth=2, label='Cascade')
plt.title('Per-Clip Inference Runtime: Direct vs Cascaded ST')
plt.xlabel('Sample index')
plt.ylabel('Runtime (sec)')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig(FIGURES_DIR / 'timing_direct_vs_cascade_35_clips.png', dpi=200)

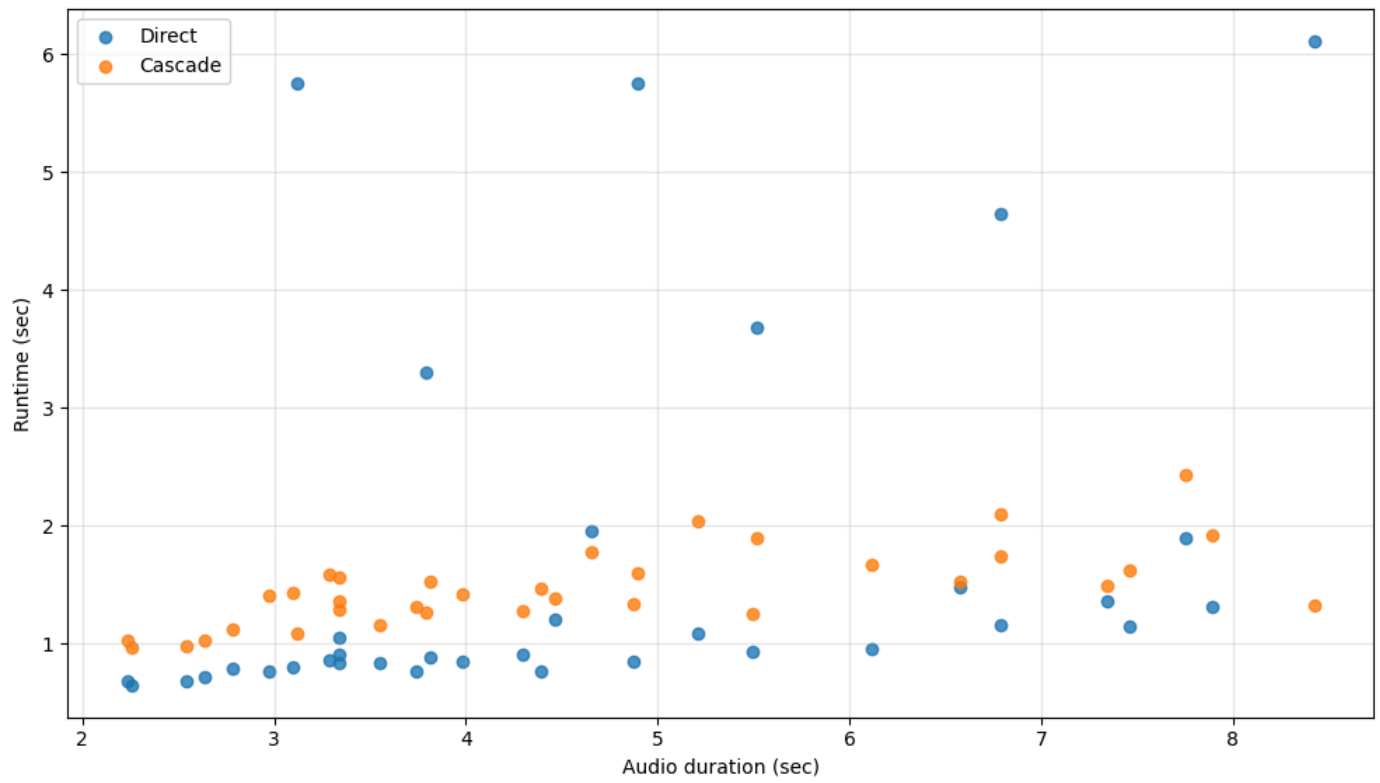
plt.figure(figsize=(10, 6))
plt.scatter(df['audio_duration_sec'], df['direct_runtime_sec'], label='Direct', alpha=0.8)
plt.scatter(df['audio_duration_sec'], df['preferred_cascade_runtime_sec'], label='Cascade', alpha=0.8)
plt.title('Inference Runtime vs Audio Duration')
plt.xlabel('Audio duration (sec)')
plt.ylabel('Runtime (sec)')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig(FIGURES_DIR / 'timing_vs_audio_duration.png', dpi=200)

plt.figure(figsize=(12, 6))
plt.bar(sample_idx, df['whisper_runtime_sec'].fillna(0), label='Whisper', alpha=0.8)
plt.bar(sample_idx, df['nllb_runtime_sec'].fillna(0), label='NLLB', alpha=0.8)
plt.title('Cascaded Pipeline Runtime Breakdown')
plt.xlabel('Sample index')
plt.ylabel('Runtime (sec)')
plt.legend()
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig(FIGURES_DIR / 'cascade_runtime_breakdown.png', dpi=200)

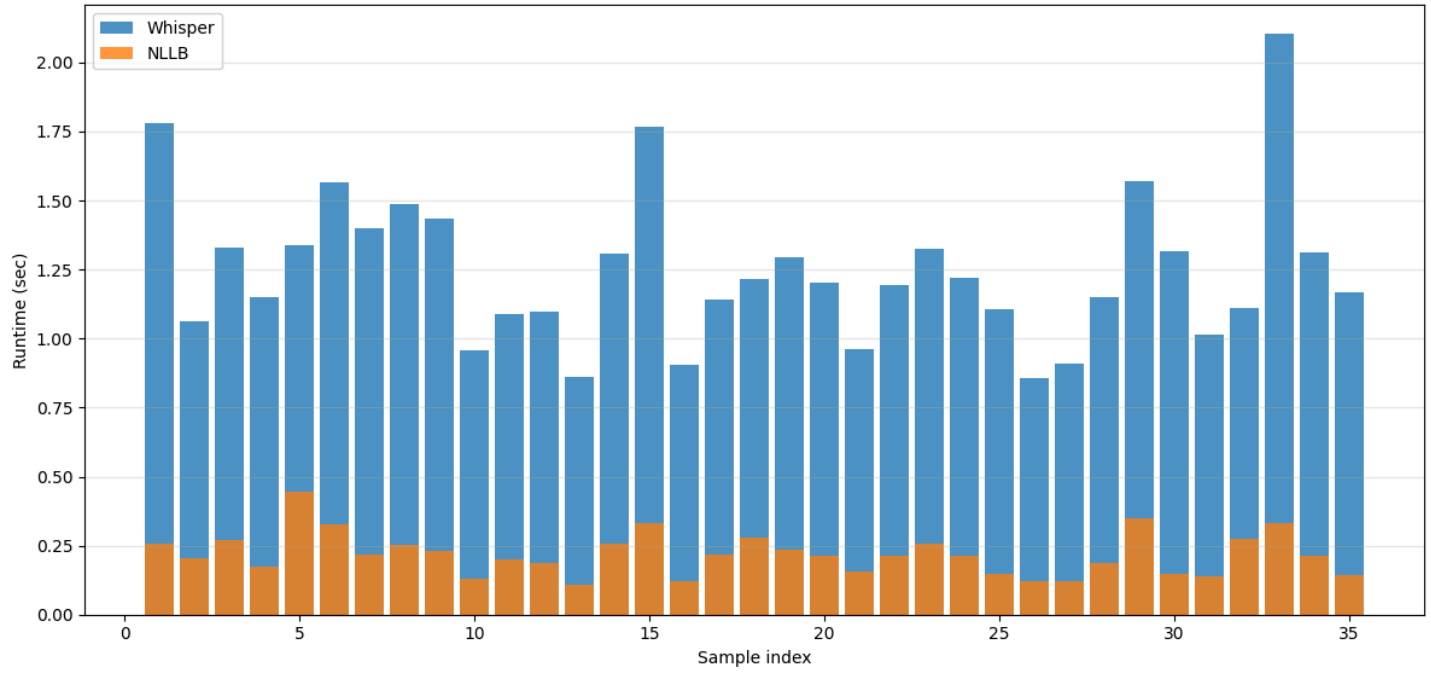
plt.figure(figsize=(12, 6))
plt.plot(sample_idx, df['direct_realtime_factor'].fillna(0), marker='o', linewidth=2, label='Direct RTF')
plt.plot(sample_idx, df['cascade_realtime_factor'].fillna(0), marker='s', linewidth=2, label='Cascade RTF')
plt.axhline(1.0, color='black', linestyle='--', linewidth=1, label='RTF = 1')
plt.title('Real-Time Factor by Clip')
plt.xlabel('Sample index')
plt.ylabel('RTF')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.savefig(FIGURES_DIR / 'realtime_factor_35_clips.png', dpi=200)
plt.show()
print(f'Saved figures to {FIGURES_DIR}')
```



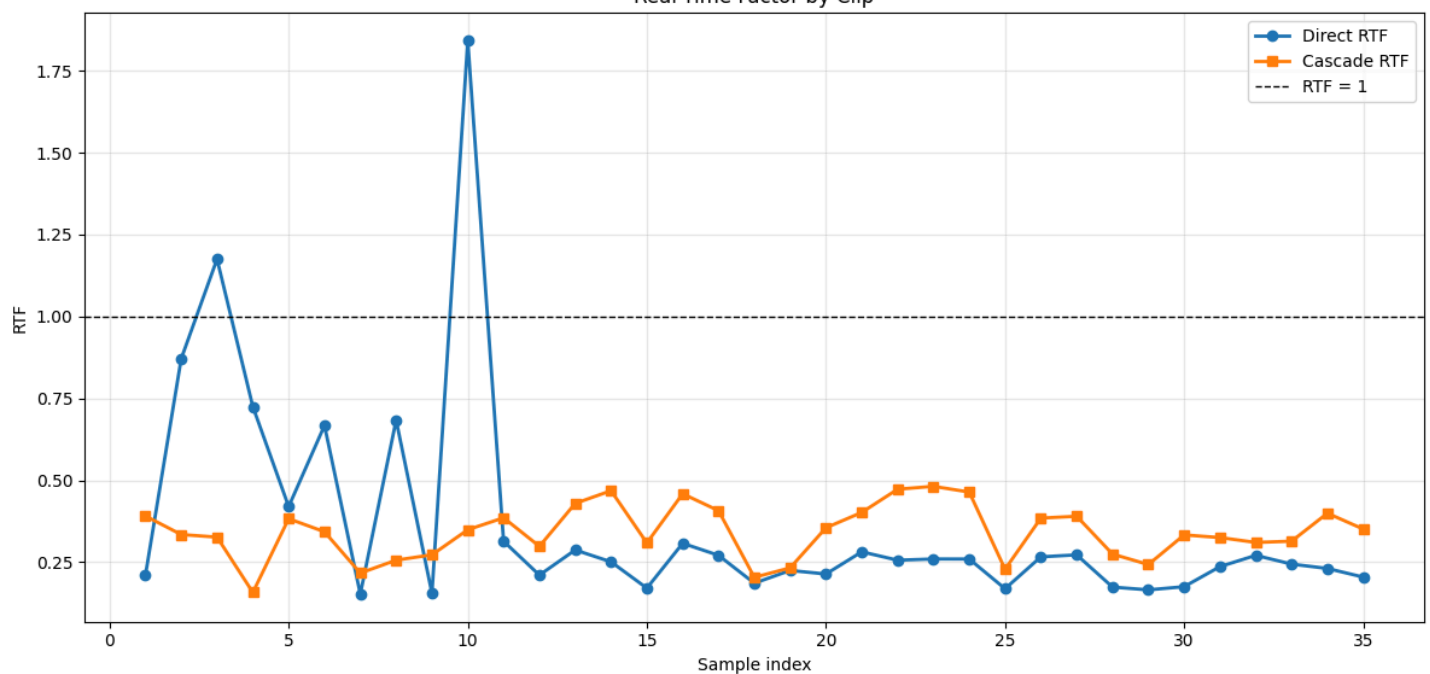
Inference Runtime vs Audio Duration



Cascaded Pipeline Runtime Breakdown



Real-Time Factor by Clip



Saved figures to /content/results/figures

```
In [15]: print('Summary of inference runtimes and real-time factors:')
```

Summary of inference runtimes and real-time factors:

15. Feasibility Conclusion

```
In [16]: print(f"1. Mean and median Direct ST per-clip runtime: {direct_runtime_values.mean():.4f}s and {direct_runtime_values.median():.4f}s")
print(f"2. Mean and median Cascaded ST per-clip runtime: {cascade_runtime_values.mean():.4f}s and {cascade_runtime_values.median():.4f}s")
print(f"3. Runtime variation across 35 clips: Direct std={direct_runtime_values.std(ddof=1):.4f}s; Cascade std={cascade_runtime_values.std(ddof=1):.4f}s")
print(f"4. Runtime relationship with audio duration: Direct corr={df['audio_duration_sec'].corr(df['direct_runtime_sec']):.3f}; Cascade corr={df['audio_duration_sec'].corr(df['cascade_runtime_sec']):.3f}")
print(f"5. Average Direct real-time factor: {direct_rtf_values.mean():.3f}")
print(f"6. Average Cascade real-time factor: {cascade_rtf_values.mean():.3f}")
print(f"7. Whisper vs NLLB timing: Whisper mean={whisper_runtime_values.mean():.3f}s; NLLB mean={nllb_runtime_values.mean():.3f}s")
print(f"8. Estimated Direct runtime for 4,200 clips: {estimated_direct_mean / 3600:.2f} hours")
print(f"9. Estimated Cascade runtime for 4,200 clips: {estimated_cascade_mean / 3600:.2f} hours")
print(f"10. Estimated combined runtime: {estimated_combined_mean / 3600:.2f} hours")

if estimated_combined_mean / 3600 <= 12:
    feasibility = 'Based on the current hardware, the full 4,200-sample experimental run appears computationally feasible within a practical wall-clock budget.'
else:
    feasibility = 'Based on the current hardware, the full 4,200-sample experiment is likely expensive and may require staged execution on multiple machines.'
print(f"11. Feasibility conclusion: {feasibility}")
print('')
print('Final feasibility statement: ' + feasibility)

summary_table = pd.DataFrame({
    'Pipeline': ['Direct', 'Cascade'],
    'Mean sec/clip': [direct_runtime_values.mean(), cascade_runtime_values.mean()],
    'Median': [direct_runtime_values.median(), cascade_runtime_values.median()],
    'Min': [direct_runtime_values.min(), cascade_runtime_values.min()],
    'Max': [direct_runtime_values.max(), cascade_runtime_values.max()],
    'Mean RTF': [direct_rtf_values.mean(), cascade_rtf_values.mean()],
    'Total 35 clips': [direct_runtime_values.sum(), cascade_runtime_values.sum()],
    'Estimated 4200 hours': [estimated_direct_mean / 3600, estimated_cascade_mean / 3600],
})
display(summary_table)
```

1. Mean and median Direct ST per-clip runtime: 1.6654s and 0.9285s
2. Mean and median Cascaded ST per-clip runtime: 1.4679s and 1.4144s
3. Runtime variation across 35 clips: Direct std=1.5770s; Cascade std=0.3341s
4. Runtime relationship with audio duration: Direct corr=0.341; Cascade corr=0.672
5. Average Direct real-time factor: 0.366
6. Average Cascade real-time factor: 0.341
7. Whisper vs NLLB timing: Whisper mean=1.249s; NLLB mean=0.219s
8. Estimated Direct runtime for 4,200 clips: 1.94 hours
9. Estimated Cascade runtime for 4,200 clips: 1.71 hours
10. Estimated combined runtime: 3.66 hours
11. Feasibility conclusion: Based on the current hardware, the full 4,200-sample experimental run appears computationally feasible within a practical wall-clock budget.

Final feasibility statement: Based on the current hardware, the full 4,200-sample experimental run appears computationally feasible within a practical wall-clock budget.

	Pipeline	Mean sec/clip	Median	Min	Max	Mean RTF	Total 35 clips	Estimated 4200 hours
0	Direct	1.665431	0.928483	0.648095	6.105438	0.365801	58.290082	1.943003
1	Cascade	1.467906	1.414431	0.968765	2.434261	0.341450	51.376710	1.712557